



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 19

Dr. Asad Mahmood

March 02, 2026

Lecture Outline

- Announcements (Assignment and Quiz 3)
- Outline of Previous Lecture:
 - **Chapter 3 – Arithmetic for Computers**
 - 3.4 Division
 - 3.5 Floating Point
- Outline to Today's Lecture:
 - **Chapter 3 – Arithmetic for Computers**
 - 3.5 Floating Point

Chapter 3

Arithmetic for Computers

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Floating Point

*“Speed gets you nowhere if
you’re headed the wrong way”*

American proverb

Floating Point

- Representation for non-integral numbers
 - Including very small and very large numbers
- Like scientific notation
 - -2.34×10^{56} ← normalized
 - $+0.002 \times 10^{-4}$ ← not normalized
 - $+987.02 \times 10^9$ ← not normalized
- In binary
 - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Types float and double in C

Floating Point Representation

- Tradeoff between precision (fraction part) and range (exponent part)
 - > Good design demands good compromise!
- Different levels of range and precision
 - Half precision (16 bits)
 - Single Precision (32 bits) – float
 - Range = 2×10^{-38} to 2×10^{38}
 - Extraordinary but not infinity -> overflow still possible!
 - And underflow!



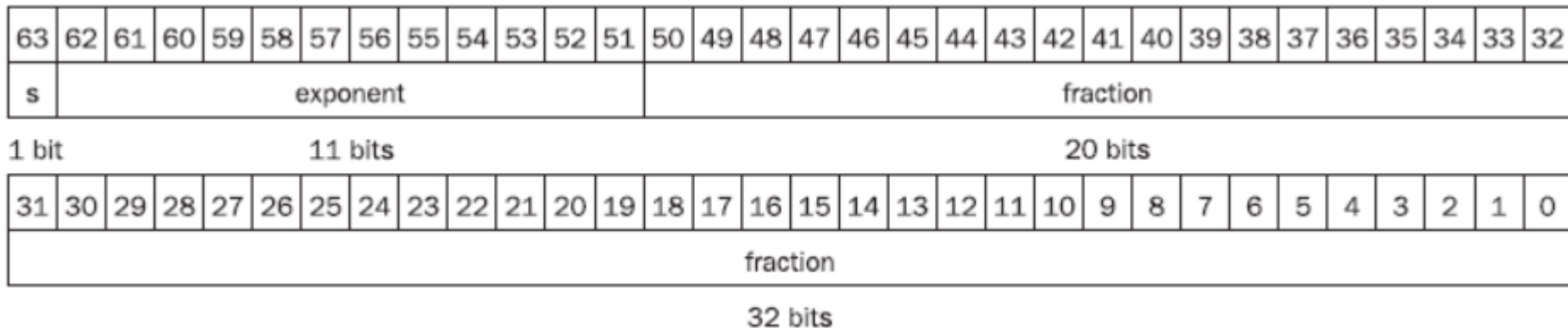
Floating Point Representation

- Different levels of precision

- Double Precision (64 bits) – double

- Range = 2×10^{-308} to 2×10^{308}

- Primary advantage is not the increased range but rather the enhanced precision!



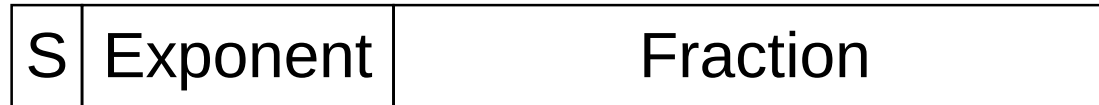
Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
 - Portability issues for scientific code
- Now almost universally adopted
- Two representations
 - Single precision (32-bit)
 - Double precision (64-bit)

IEEE Floating-Point Format

single: 8 bits
double: 11 bits

single: 23 bits
double: 52 bits



- Equivalent decimal number
- Normalize significand:
 - $1.0 \leq |\text{significand}| < 2.0$
- Exponent: biased representation: actual exponent + Bias
 - Ensures exponent is unsigned
 - Single: Bias = 127; Double: Bias = 1203

IEEE Floating-Point Format

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1–254	Anything	1–2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

- Exponents 00000000 and 11111111 reserved
- Denormalized numbers are very small nos. between the smallest normalized/normal no. and 0.
- Infinity vs Not-a-number (NaN)
- Smallest value ?
- Largest value ?

Floating-Point Example

- Example 1: Represent -0.75 in single precision format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
1 bit									8 bits								23 bits																

- Example 2: What decimal number is represented by the following single precision number?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Floating-Point Precision

- Precision = No. of significant digits accurately represented
- For floating points
 - Single Precision
 - 23 fraction bits + 1 = 24 bits can be accurately represented -> 24 bit binary precision
 - Equivalent decimal precision ?
 - Double Precision
 - Binary precision ?
 - Equivalent decimal precision ?

Floating-Point Addition

- How would we add decimal nos. in scientific notation?
- Process followed:
 1. Align decimal points
 - Shift number with smaller exponent
 2. Add significands
 3. Normalize result & check for over/underflow
 4. Round and renormalize if necessary

Floating-Point Addition

- Same process to follow for binary floating point addition:
 - Example: $(1.000_2 \times 2^{-1}) + (-1.100_2 \times 2^{-2})$
1. Align binary points
 2. Add significands
 3. Normalize result & check for over/underflow
 4. Round and renormalize if necessary